StudyAI

AI Flashcard Engine • Build Challenge Submission

Problem 1: The Flashcard Engine | Submitted by Abhishek Pathak

Live URL	GitHub Repository
https://ai-study-tool-wyrb.vercel.app/	github.com/Codeabhi096/AI-Study-Tool

1. What I Built

I built StudyAI — an AI-powered study assistant that transforms any document or pasted text into a complete, multi-format learning experience. The core belief behind this project mirrors Cuemath's own: passive re-reading is the least effective way to study. Active recall, not re-consumption, is what builds real retention.

A student drops in a PDF — a chapter on quadratic equations, notes on the French Revolution, a dense biology handout — and within seconds they have five different learning tools generated from the same content:

Mode	Output	What it does for the student
Explain	Plain English breakdown	Converts dense academic language into simple, clear explanations a student actually understands
Summarize	Structured summary	Pulls out the key ideas, relationships, and concepts — without the noise
Quiz Me	MCQ-style quiz	Forces active recall with questions that test real understanding, not just recognition
Flashcards	Flip-card deck	Interactive front/back cards for spaced repetition practice, fully navigable
Mind Map	Visual concept map	Shows how ideas connect — great for subjects with lots of relationships (history, biology, etc.)

The app supports multiple input formats — PDF upload, plain text paste, TXT, CSV, and JSON — and has been deployed live on Vercel (frontend) and Render (backend). It also supports multi-language output: English, Hindi, Hinglish, Spanish, and French.

2. Key Decisions & Tradeoffs

2.1 Architecture: Decoupled Frontend + Backend

The biggest early decision was whether to build a monolith or a decoupled architecture. I chose a split: a static HTML/CSS/JS frontend deployed on Vercel, and a FastAPI Python backend deployed on Render.

Decision Area	Choice Made	Reasoning
Architecture	Decoupled (Vercel + Render)	Keeps AI key server-side; frontend can be CDN-cached for speed
Backend Framework	FastAPI (Python)	Native PDF/file parsing libraries, fast async, clean API design
Frontend Stack	Vanilla HTML/CSS/JS	Zero build complexity, instant deploy, full control over UX
AI Provider	Gemini API	Generous free tier for prototyping; easy swap to Claude/GPT if needed
Deployment	Vercel + Render	Both have free tiers; Vercel excels at static, Render for Python APIs
Containerization	Docker (Dockerfile included)	Makes backend portable across any hosting platform

2.2 Flashcard Quality vs. Speed

The challenge brief specifically called out the difference between lazy 10-card shallow decks and great teacher-written cards. I invested heavily in the AI prompt design for flashcard generation. The prompt instructs the model to:

- Cover key concepts, definitions, cause-effect relationships, and edge cases — not just surface facts
- Write cards in a question-answer format that forces active recall, not passive recognition
- Vary difficulty: basic recall cards alongside deeper application and synthesis cards
- Use the student's own language register — not copy-pasted academic prose

The tradeoff here was depth vs. speed. A richer prompt takes longer and costs more API tokens. I chose depth — a 2-second wait for genuinely useful cards beats instant but shallow output.

2.3 Mind Map as a Visual Learning Layer

Most flashcard tools stop at cards. I added a mind map mode specifically because certain subjects — history, biology, systems thinking — are better understood through relationships than through linear lists. This was an intentional design choice beyond the minimum spec, because I believe the best study tools are multi-modal: they match the learning task to the right format.

2.4 Language Support

Cuemath operates pan-India with a user base that spans multiple languages and comfort levels with English. I built in output language selection (Hindi, Hinglish, Spanish, French) from day one — not as an afterthought. When a student gets an explanation in Hinglish, the cognitive load drops dramatically and comprehension rises. This was a deliberate product decision, not a technical flex.

2.5 Security: API Key Isolation

All AI API credentials live server-side in Render environment variables. The frontend never touches the key — it makes requests to the backend, which proxies to the AI provider. This satisfies the challenge's hard requirement on API key safety and is also simply the right production pattern.

3. Tech Stack

Layer	Technology	Purpose
Frontend	HTML / CSS / JS	UI, flashcard flip interactions, mind map rendering
Backend	Python + FastAPI	File parsing, AI prompt orchestration, REST API
AI Layer	Gemini API	All content generation — explain, summarize, quiz, flashcards, mindmap
PDF Parsing	PyMuPDF / pdfplumber	Text extraction from uploaded PDFs
Deployment (FE)	Vercel	Static site hosting with CDN
Deployment (BE)	Render	Python backend hosting with env variable support
Containerization	Docker	Portable backend deployment via Dockerfile
Version Control	GitHub	Source code and commit history

4. What I'd Improve With More Time

4.1 True Spaced Repetition (SM-2 Algorithm)

The current build generates flashcard decks and lets you flip through them. What's missing is intelligence about which cards to show when. The SM-2 spaced repetition algorithm — used by Anki — calculates review intervals based on how hard you rated each card. With more time, I would implement:

- A per-card difficulty rating (Easy / Hard / Forgot) after each flip
- SM-2 interval calculation to schedule when each card comes up next
- A daily review queue showing only cards due today
- Local persistence (IndexedDB or a lightweight backend DB) to remember your history

4.2 Mastery Tracking & Progress Dashboard

Students need to feel progress. I would add a per-deck mastery view: a visual breakdown showing which cards are in the 'new', 'learning', and 'mastered' buckets, with a streak counter and a review calendar (similar to GitHub's contribution graph). This is what turns a utility into a habit.

4.3 Deck Management & Library

Right now, each session is stateless — you start fresh. With a persistent backend (PostgreSQL on Render, for instance), I would add a full deck library: named decks, browse by subject, pick up where you left off, search across all cards, and share decks. This is the difference between a demo and a product.

4.4 Richer Flashcard Formats

The current cards are text-only. For subjects like math, chemistry, or geography, you want:

- LaTeX rendering for equations
- Image-based cards (upload a diagram, get labeled cards)
- Cloze deletion — fill-in-the-blank style — which is proven to boost recall more than simple Q&A

4.5 Mobile-First Responsive Design

Most students study on their phones. The current build is functional on mobile but not optimized for it. I would invest in a proper mobile layout, swipe gestures for card navigation, and possibly a PWA wrapper for offline use.

5. Interesting Challenges & How I Solved Them

Challenge 1: Getting AI to Generate Genuinely Useful Cards

The first version of the flashcard prompt produced shallow, surface-level cards — exactly the 'scraped by a bot' output the brief warned against. The problem was that the prompt was too open-ended.

Solution: I iterated on the system prompt extensively, adding explicit instructions about card depth (key concepts, definitions, cause-effect, edge cases), card format (question on front, full explanation on back, not just a one-word answer), and card variety (mix of recall, application, and synthesis questions). The quality jumped significantly after this.

Challenge 2: PDF Text Extraction Inconsistencies

PDFs are notoriously messy. Scanned PDFs, PDFs with complex layouts, or tables-heavy documents produced garbled text that confused the AI model. Multi-column layouts especially were a problem — text was being read left-to-right across columns instead of column by column.

Solution: I added a preprocessing step that normalizes extracted text — strips header/footer artifacts, fixes line breaks, and handles encoding issues before the content reaches the AI layer. For highly malformed inputs, I surface a warning to the user rather than silently generating bad cards.

Challenge 3: Cold Start Latency on Free Hosting

Render's free tier spins down the backend after inactivity. First-request latency after a cold start can be 10-30 seconds, which is a jarring user experience.

Solution: I added a loading state UI with clear messaging ('Warming up the AI engine...') so users know something is happening. I also added a lightweight health-check ping from the frontend on page load to wake the backend before the user submits their first request — reducing perceived wait time.

Challenge 4: Mind Map Layout for Complex Documents

Automatically generating a readable mind map from unstructured text is genuinely hard. Early attempts produced flat, over-crowded graphs or missed the hierarchical structure of the content entirely.

Solution: I structured the AI prompt to first identify the central topic, then extract 4-6 primary branches, then 2-3 sub-nodes per branch — mirroring how a human would build the map manually. The structured JSON output from the AI is then rendered as an interactive node-link diagram in the browser.

6. Alignment With the Challenge Brief

Brief Requirement	Status	Implementation Notes
PDF → Flashcard conversion	☑ Done	PDF upload, text extraction, AI-generated cards
Ingestion quality — deep cards	☑ Done	Prompt-engineered for concepts, defs, relationships, edge cases
Spaced repetition logic	⚠ Partial	Card flip UI done; SM-2 scheduling is the next build priority
Progress & mastery tracking	⚠ Partial	Session-level feedback done; persistent mastery is next
Deck management	☐ Planned	Stateless for now; full deck library planned with DB backend
Delight & design	☑ Done	Clean UI, smooth animations, multi-mode output, language switch
Deployed & publicly accessible	☑ Done	Live on Vercel — click and use immediately
No exposed API keys	☑ Done	All keys server-side in Render env variables; none in frontend
Multiple input formats	☑ Done	PDF, TXT, CSV, JSON, paste — all supported
GitHub repo (public)	☑ Done	github.com/Codeabhi096/AI-Study-Tool

The best study tool is the one students actually use.

StudyAI is built around that principle — not just to generate content, but to make the act of studying less painful and more effective.

ai-study-tool-wyrb.vercel.app | **github.com/Codeabhi096/AI-Study-Tool**